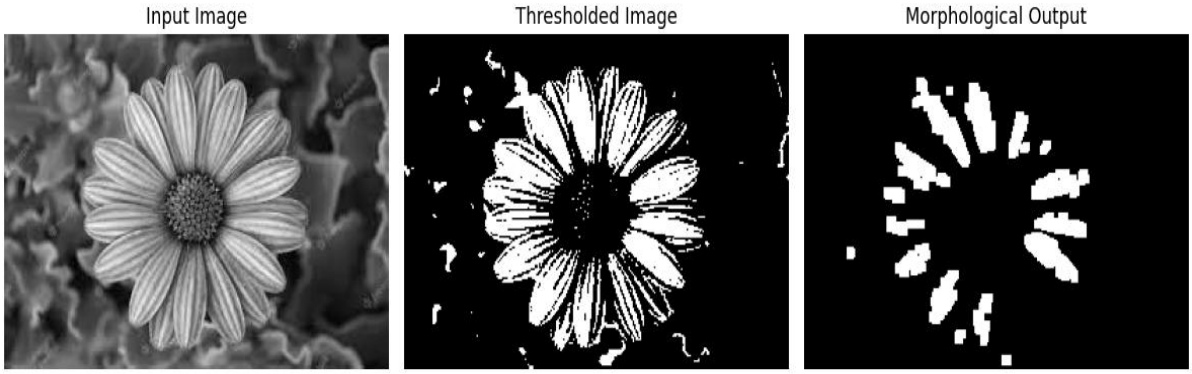
EXP 16

```python
from google.colab import files
import cv2
import matplotlib.pyplot as plt
uploaded = files.upload()
filename = list(uploaded.keys())[0]
img = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)
if img is None:
    print("Error: Unable to read image")
else:
    _, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
    opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 3, 1)
    plt.imshow(img, cmap='gray')
    plt.title("Input Image")
    plt.axis("off")
    plt.subplot(1, 3, 2)
    plt.imshow(binary, cmap='gray')
    plt.title("Thresholded Image")
    plt.axis("off")
    plt.subplot(1, 3, 3)
    plt.imshow(opening, cmap='gray')
    plt.title("Morphological Output")
    plt.axis("off")
    plt.tight_layout()
    plt.show()
```

OUTPUT



EXP 17

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
X = np.array([
    [1, 1], [2, 2], [2, 1],
    [6, 6], [7, 8], [8, 7]
])
y = np.array([0, 0, 0, 1, 1, 1])
model = LogisticRegression()
model.fit(X, y)
pred = model.predict(X)
print("Input Data:")
print(X)
print("\nActual Classes:", y)
print("Predicted Classes:", pred)
print("Accuracy:", model.score(X, y))
plt.scatter(X[:,0], X[:,1], c=y, cmap="coolwarm", s=80)
x_values = np.linspace(0, 9, 100)
w = model.coef_[0]
b = model.intercept_[0]
y_values = -(w[0] * x_values + b) / w[1]
plt.plot(x_values, y_values)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Linear Separability using Logistic Regression")
plt.show()
```

OUTPUT

Input Data:

[[1 1]

[2 2]

[2 1]

[6 6]

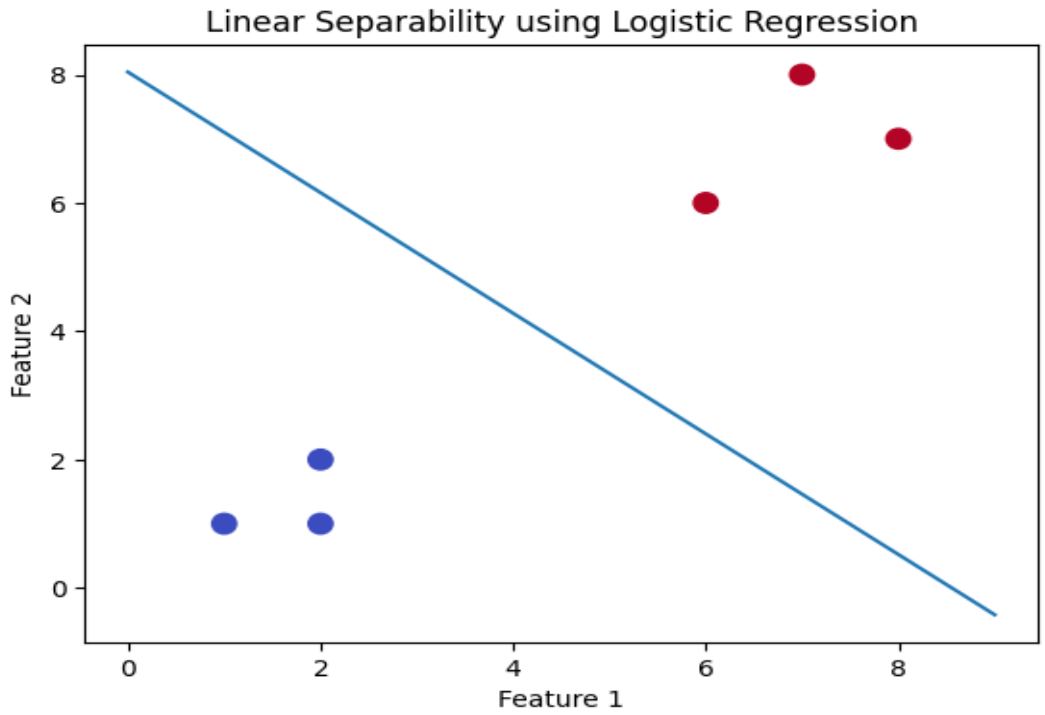
[7 8]

[8 7]]

Actual Classes: [0 0 0 1 1 1]

Predicted Classes: [0 0 0 1 1 1]

Accuracy: 1.0

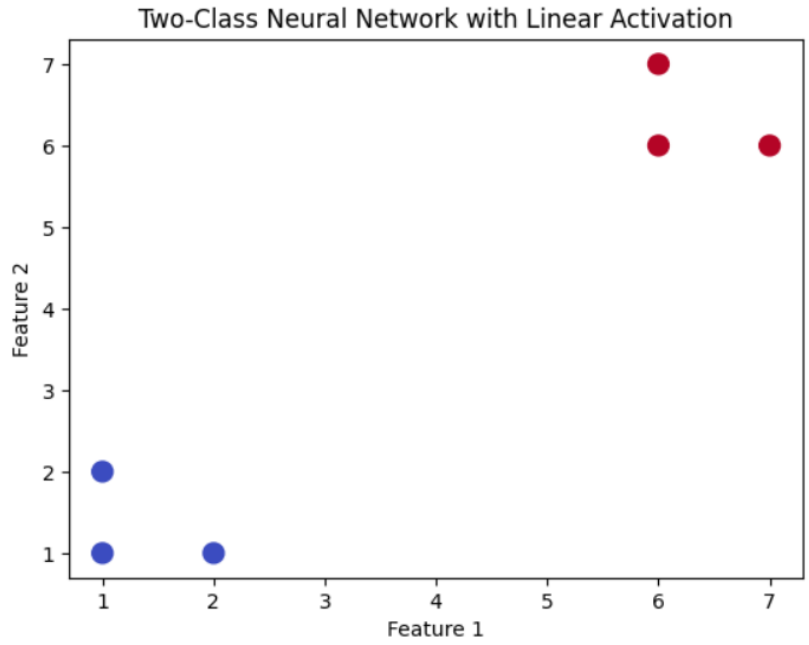


EXP 18

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
X = np.array([
    [1, 1], [2, 1], [1, 2],
    [6, 6], [7, 6], [6, 7]
])
y = np.array([0, 0, 0, 1, 1, 1])
model = MLPClassifier(
    hidden_layer_sizes=(5,),
    activation='identity',
    max_iter=2000,
    random_state=42
)
model.fit(X, y)
pred = model.predict(X)
print("Actual Classes:", y)
print("Predicted Classes:", pred)
print("Accuracy:", model.score(X, y))
plt.scatter(X[:,0], X[:,1], c=y, cmap="coolwarm", s=100)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Two-Class Neural Network with Linear Activation")
plt.show()
```

OUTPUT

```
Actual Classes: [0 0 0 1 1 1]
Predicted Classes: [0 0 0 1 1 1]
Accuracy: 1.0
```

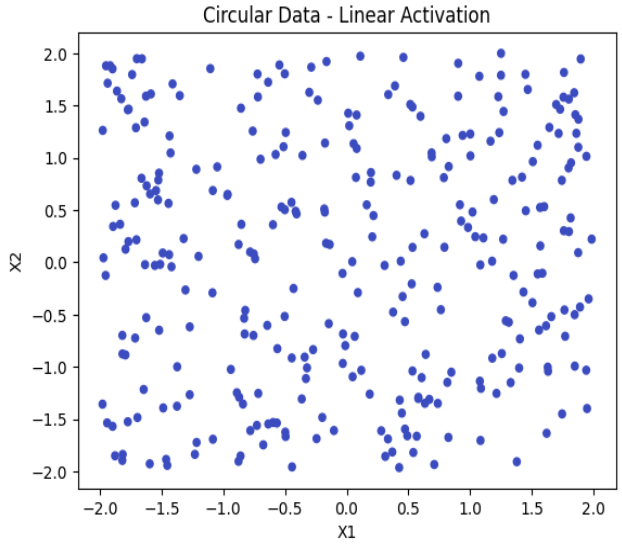


EXP 19

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
np.random.seed(42)
X = np.random.uniform(-2, 2, (300, 2))
r = np.sqrt(X[:,0]**2 + X[:,1]**2)
y = (r > 1).astype(int)
model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation='identity',
    max_iter=2000,
    random_state=42
)
model.fit(X, y)
pred = model.predict(X)
print("Accuracy:", model.score(X, y))
plt.scatter(X[:,0], X[:,1], c=pred, cmap="coolwarm", s=20)
plt.xlabel("X1")
plt.ylabel("X2")
plt.title("Circular Data - Linear Activation")
plt.show()
```

OUTPUT

Accuracy: 0.8333333333333334



EXP 20

CODE

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
X = np.array([
   [1,1], [1,2], [2,1],
   [5,5], [5,6], [6,5],
   [9,1], [9,2], [8,1]
])
y = np.array([
   0,0,0,
   1,1,1,
   2,2,2
])
model = MLPClassifier(
   hidden_layer_sizes=(10,),
   activation='identity',
   max_iter=3000,
   random_state=42
)
model.fit(X, y)
pred = model.predict(X)
print("Actual Classes:")
print(y)
print("\nPredicted Classes:")
```

```
print(pred)
print("\nAccuracy:", model.score(X, y))
plt.scatter(
   X[:,0],
   X[:,1],
   c=y,
   cmap="viridis",
   s=100
)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Multi-Class Neural Network with Linear Activation")
plt.show()
```

OUTPUT

Actual Classes:

[0 0 0 1 1 1 2 2 2]

Predicted Classes:

[0 0 0 1 1 1 2 2 2]

Accuracy: 1.0